Machine Learning Lectures

Data Preparation

Data Preparation

By:
Dr. Andrey Holz
https://www.linkedin.com/in/andrey-holz

Addressing Key Challenges in EDA

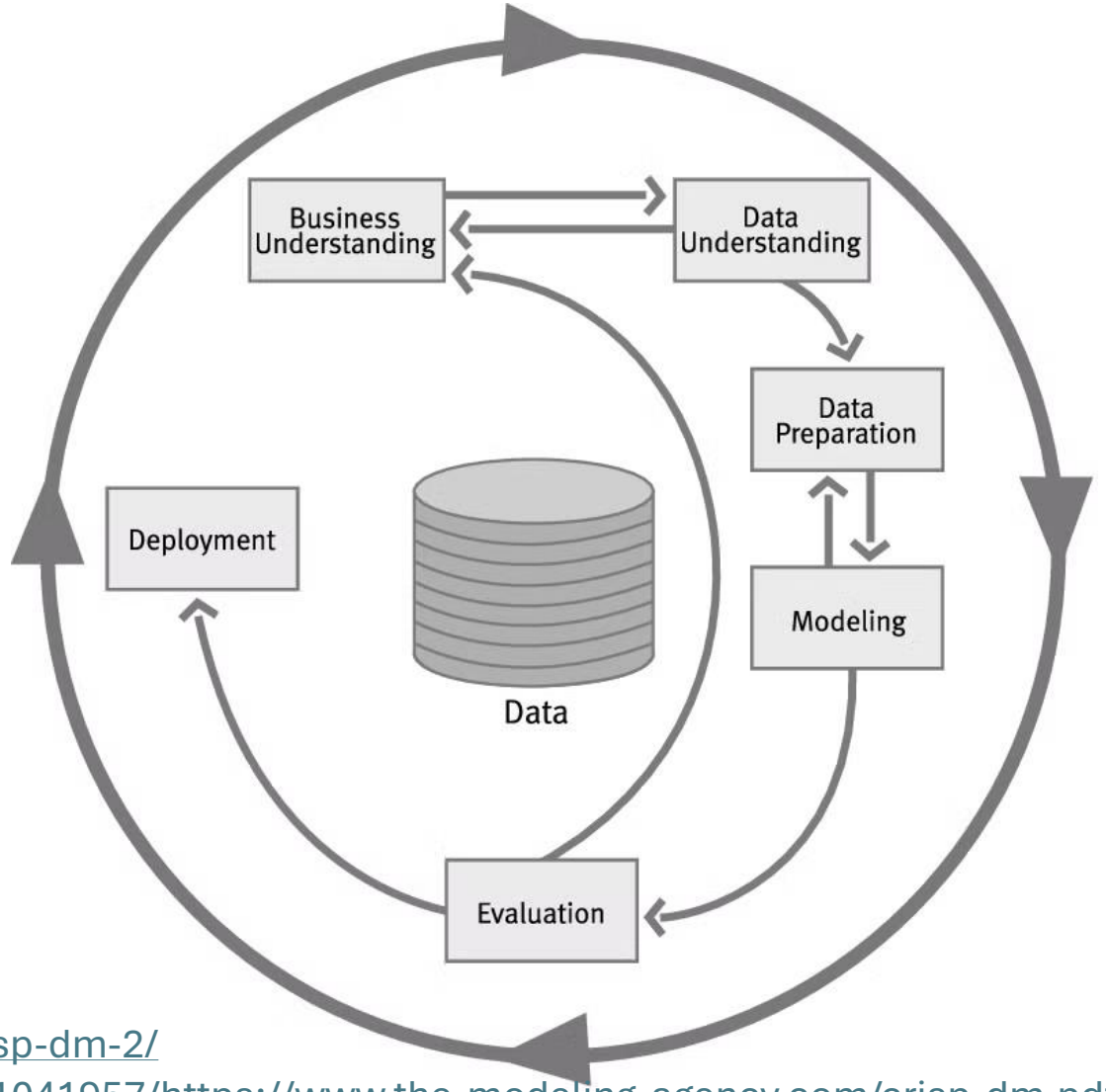
Previously on EDA Lecture:

- **Dealing with Missing Data:**
  - Missing data can skew results and affect model performance.
- **Managing Inconsistent Data:**
  - Inconsistent formats (e.g., date formats, units) cause errors in analysis.
- **Identifying Outliers:**
  - Outliers can distort analysis and affect the accuracy of models.
- **Handling High Dimensionality:**
  - Too many variables can lead to **overfitting** and make interpretation difficult.
- **Addressing Data Imbalance:**
  - Imbalanced data (e.g., one class dominating) can result in biased models.

Agenda

1. **Introduction to Data Preparation**
2. **Data Cleaning (Missing Values, Duplicates, Outliers)**
3. **Transformation and Encoding**
4. **Data Preparation Challenges**
5. **Summary and Wrap-Up**

Data Analytics pipeline: CRISP-DM



https://www.datascience-pm.com/crisp-dm-2/
https://web.archive.org/web/20220401041957/https://www.the-modeling-agency.com/crisp-dm.pdf

Business Understanding

Determine Business Objectives
Background
Business Objectives
Business Success Criteria

Assess Situation
Inventory of Resources
Requirements, Assumptions, and Constraints
Risks and Contingencies
Terminology
Costs and Benefits

Determine Data Mining Goals
Data Mining Goals
Data Mining Success Criteria

Produce Project Plan
Project Plan
Initial Assessment of Tools and Techniques

Data Understanding

Collect Initial Data
Initial Data Collection Report

Describe Data
Data Description Report

Explore Data
Data Exploration Report

Verify Data Quality
Data Quality Report

Data Preparation

Select Data
Rationale for Inclusion/ Exclusion

Clean Data
Data Cleaning Report

Construct Data
Derived Attributes
Generated Records

Integrate Data
Merged Data

Format Data
Reformatted Data

Dataset
Dataset Description

Modeling

Select Modeling Techniques
Modeling Technique
Modeling Assumptions

Generate Test Design
Test Design

Build Model
Parameter Settings
Models
Model Descriptions

Assess Model
Model Assessment
Revised Parameter Settings

Evaluation

Evaluate Results
Assessment of Data Mining Results w.r.t. Business Success Criteria
Approved Models

Review Process
Review of Process

Determine Next Steps
List of Possible Actions
Decision

Deployment

Plan Deployment
Deployment Plan

Plan Monitoring and Maintenance
Monitoring and Maintenance Plan

Produce Final Report
Final Report
Final Presentation

Review Project
Experience Documentation

Introduction to Data Preparation

Data Preparation: Building a Solid Foundation

Why It Matters:

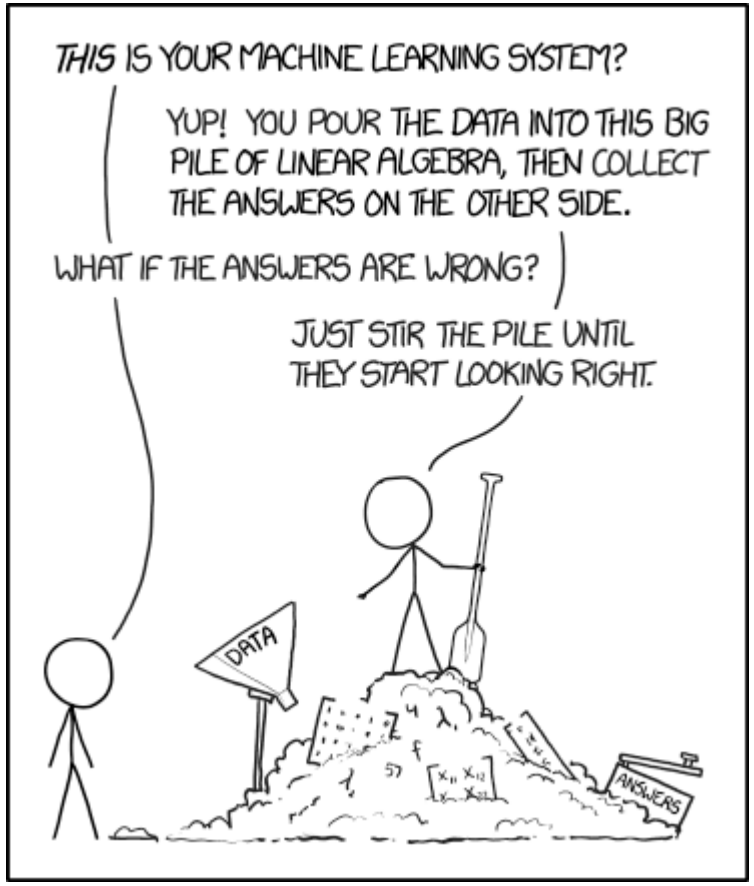
Data preparation takes a significant amount of time.

Garbage In -> Garbage Out

Key Techniques:

Data cleaning, Transformation & Encoding.

<u>Your model is only as good as the data fed into it.</u>

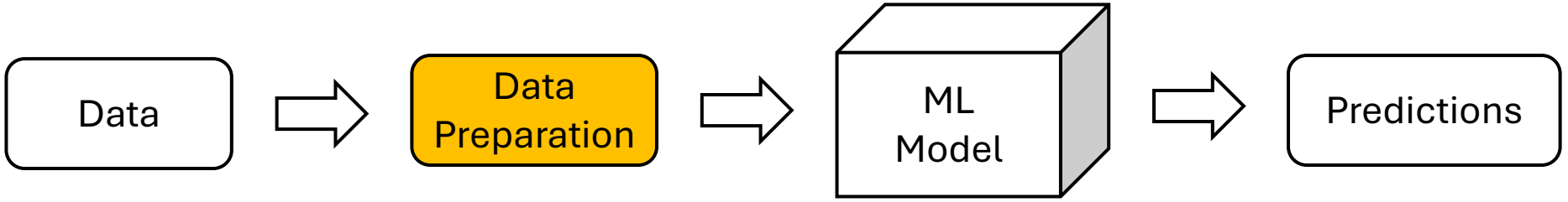


https://xkcd.com/1838/

Importance of Data Preparation

"Why is Data Preparation Crucial?"

- Data preparation is the foundation for machine learning models.
- **Garbage In, Garbage Out:** Poor quality data results in poor model performance.



- *Real-world examples:*
  - *Healthcare: misdiagnosis due to missing data*
  - *Finance: biased results due to outliers.*

Data Preparation Steps

#	§	Step	Description
1		**Data Collection**	Gathering raw data from various sources.
2	I	**Data Cleaning**	Handling missing values, duplicates, and outliers.
3	I	**Transformation and Encoding**	Scaling, normalizing, and encoding categorical features.
4	II	**Feature Selection and Feature Generation**	Selecting important features and creating new ones.
5	II	**Dimensionality Reduction**	Reducing the number of features to simplify the model.
6	III	**Data Splitting**	Dividing data into training, validation, and test sets.
7	III	**Cross-Validation**	Validating the model's robustness by splitting the training set multiple times.
8	III	**Data Leakage Prevention**	Ensuring no information from the test set leaks into the training set.
9	III	**Building Data Pipelines**	Automating the data preparation process.

Will be covered today

Data Cleaning

General Overview of Data Cleaning

- **Main Goal:** Ensuring data is free of errors, inconsistencies, and inaccuracies.
- **Key Processes:**
  - Handling missing values,
  - Removing duplicates,
  - Correcting inconsistencies.
- **Why It Matters:** Clean data leads to more accurate models and better decision-making.

Understanding **Missing Values**

Source of Missing Values:
Data entry errors, system issues, or subjects opting out of answering questions.

Types of Missing Values:

- Missing Completely at Random (**MCAR**):
  - The probability of a value being missing is the same for all observations.
    Example: survey responses are missing because some participants randomly chose not to answer
- Missing at Random (**MAR**) or Missing Conditionally at Random
  - The probability of missingness is related to some of the observed data but not to the missing data itself.
    Example: older individuals are less likely to report their income in a survey, the missingness depends on the observed characteristic (age) but not the income value itself.
- Missing not at Random (**MNAR**)
  - The probability of missingness is related to the value itself.
    Example: People with higher incomes are less likely to report their income,

Handling Techniques: Deletion or Imputation

Methods for Handling **Missing Values**

Imputation

Time-Series Data

No Trend & no Seasonality	Mean, Median, Mode, Random Sample Imputation
No Trend & Seasonality	Seasonal Adjustment + Interpolation
Trend & no Seasonality	Linear Interpolation
Trend & Seasonality	Seasonal Adjustment + Interpolation

General Data

Categorical	Make NA as level, Logistic Regression, ML, Multiple Imputation
Numerical	Mean, Median, Mode, Multiple Imputation, Linear Regression, ML

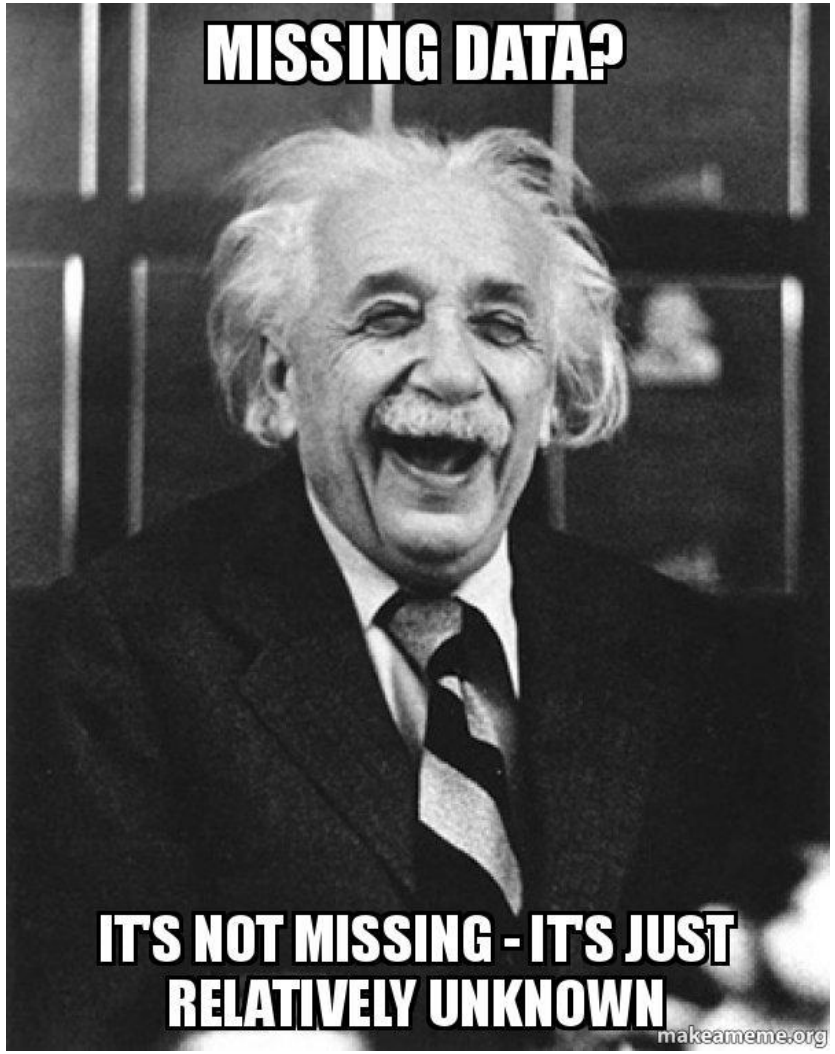
Deletion

- Listwise
- Pairwise
- Drop column

id	gender	age	result
1	Male	20	Positive
2	Female	—	Negative
3	Female	30	Positive
4	—	28	Negative
5	Female	—	Positive
6	Male	25	Positive
7	Male	21	Positive

Missing Values Imputing Techniques

- Time-Series
  - Last Observation Carried Forward (**LOCF**)
    & Next Observation Carried Backward (**NOCB**)
  - Linear Interpolation
  - Seasonal Adjustment + Linear Interpolation
- Not Time Series
  - Mean, Median, Mode, Constant
- Model-based:
  - Linear Regression
  - Logistic Regression
  - KNN
  - Multiple Imputation
  - Neural Networks



Live Coding & Demos #1

Dealing with **Duplicates**

- **Definition:** Records that appear more than once in the dataset.
- **Bias from duplicates:** Extra weight given to repeated records, skewing results.
- **Techniques:** Removing duplicates based on unique identifiers OR on complete match of features & analytics.

Detecting and Handling Outliers

Definition: data points that differ significantly from other observations. They may be a result of variability in the data or indicate errors.

Examples:

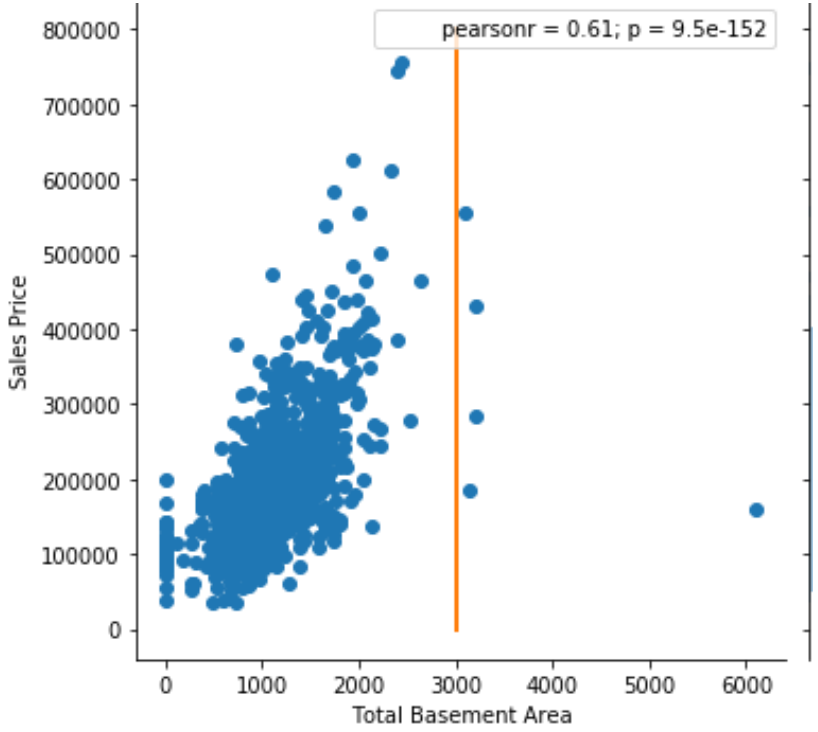
- High salaries of CEOs vs. average employees.
- Anomalous spikes in sensor data or stock prices.

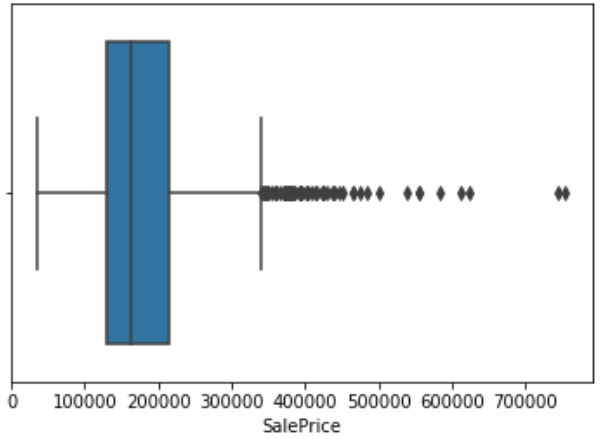
Outliers can significantly distort statistical analyses and impact models, especially those that are **sensitive to scale**, such as **linear regression** and **KNN**.

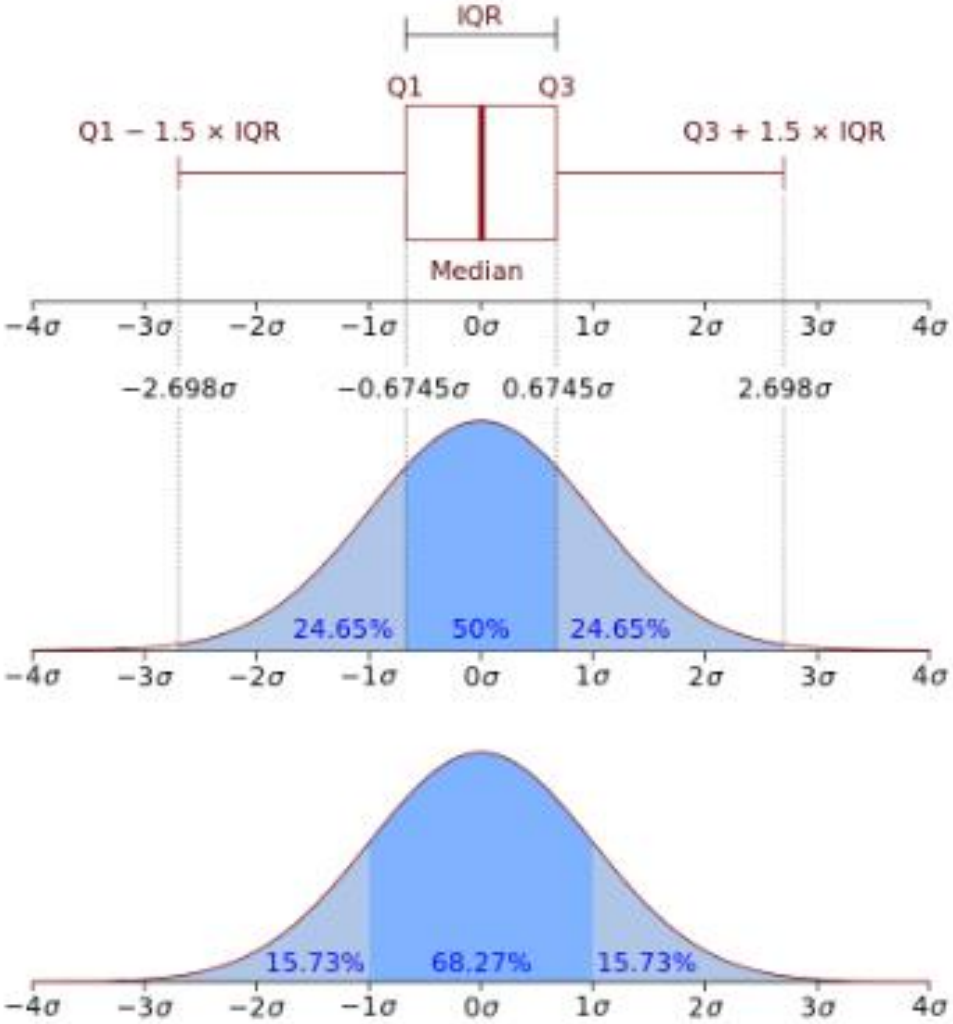
Traditional Outlier Detection Methods

- **Scatter Plot**
  - Visual tool to identify outliers based on location and distance
- **Boxplots**:
  - Visual tool using quartiles to identify outliers beyond whiskers.
- **IQR (Interquartile Range)**:
  - Measures the spread of the middle 50% of data. Outliers are outside 1.5 * IQR from Q1 or Q3.
- **Z-Score**:
  - $\boldsymbol{Z_{score} = \frac{X - \mu}{\sigma}}$
  - How many **std**'s a data point is from the mean. Common threshold: Z > 3 or Z < -3.
- **Cook's distance:**
  - Identifies influential points in **regression models** that may disproportionately affect the model.
  - $\boldsymbol{D_i = \frac{\sum_{j=1}^{n}(\hat{y}_j - \hat{y}_{j(i)})^2}{ps^2}}$, where $\boldsymbol{s^2 = \frac{e^T e}{n-p}}$, p is the rank of the model (i.e., number of independent variables in the design matrix) and $\boldsymbol{\hat{y}_{j(i)}}$ is the fitted response value obtained when excluding i, $\boldsymbol{s^2}$ is the mean squared error of the regression model

Traditional Outlier Detection Methods





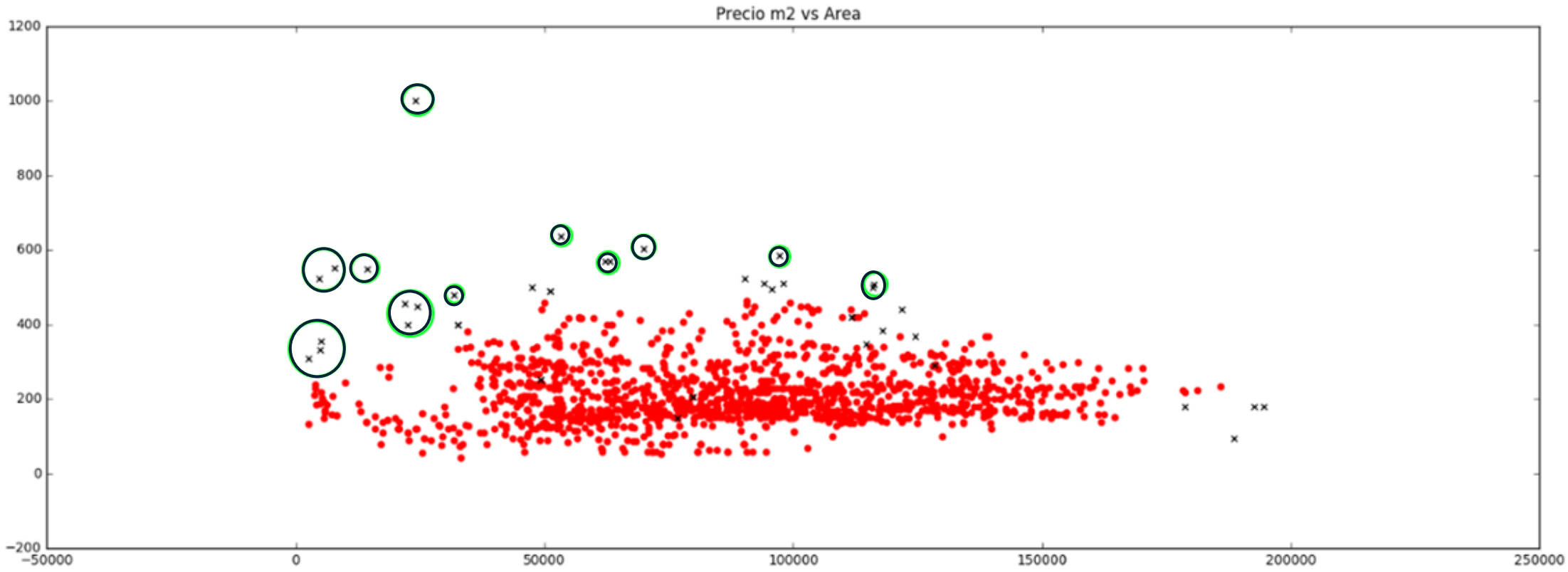


Advanced Outlier Detection Techniques

- **K-Means**: Outliers are points far from cluster centroids.
- **DBSCAN**: Detects outliers in low-density regions where no clusters form.
- **Isolation Forest**: Identifies outliers by isolating them with fewer splits in decision trees.
- **Local Outlier Factor (LOF)**: Detects outliers by comparing the local density of a point to its neighbors.

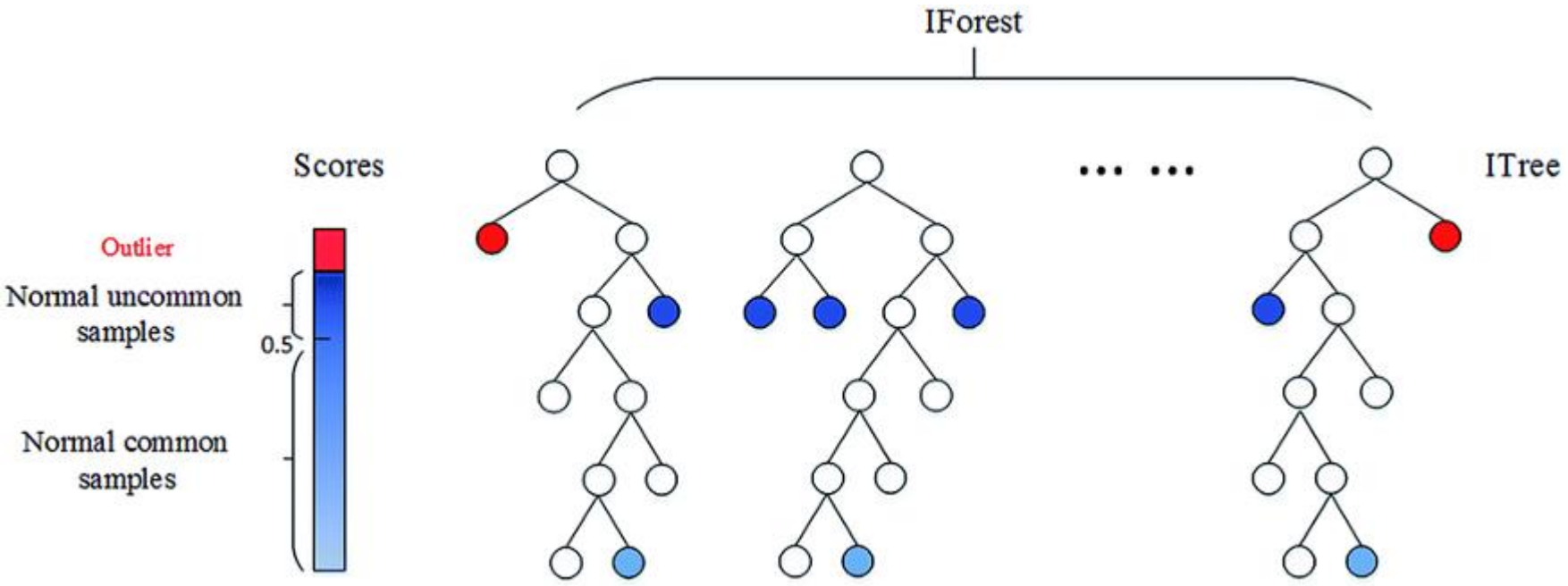
Outlier Detection with DBSCAN

- All points within the cluster are mutually density-connected.
- If a point is density-reachable from any point of the cluster, it is part of the cluster as well.



Outlier Detection with Isolation Forest

Basic principle: outliers are few and far from the rest of the observations



Deep Learning-Based Techniques

- **Autoencoders**:
  - Compress the data into a lower-dimensional space. Data with high reconstruction errors are likely outliers.

- **LSTM for Time-Series**:
  - Predicts the next value in the series; deviations from predicted trends are outliers.

Time-Series and Multivariate Outlier Detection

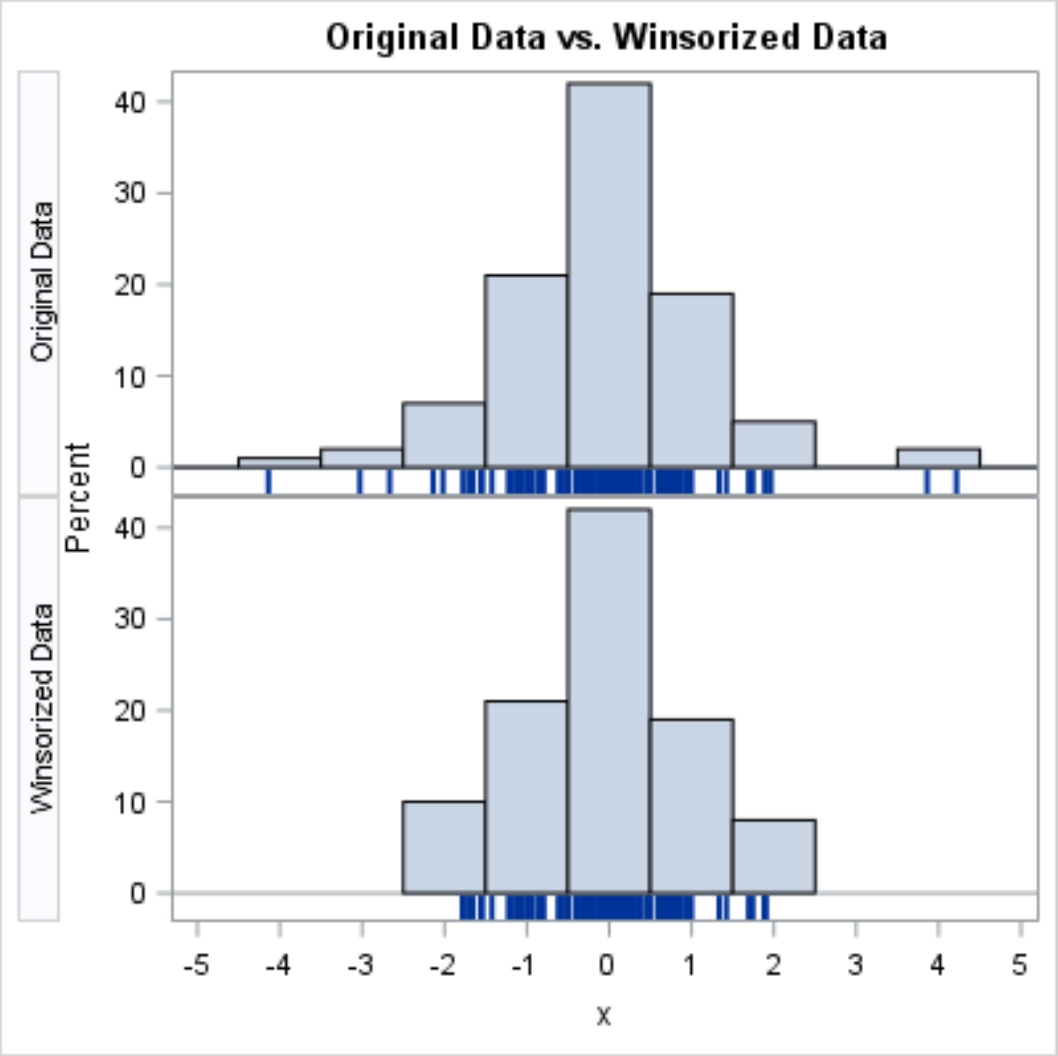
- **ARIMA**: Detect outliers by comparing actual vs. predicted values.
- **Spectral Residuals**: Detect anomalies in the frequency domain.
- **PCA & Mahalanobis Distance**: Detect anomalies in the frequency domain.
- **Hybrid Approaches**: Combining autoencoders with ARIMA or other models for better time-series anomaly detection.

Handling Outliers After Detection

- **Remove:** Effective when the data is incorrect or irrelevant
- **Impute like missing values:**
  - **Winsorize** (**cap**) values: Useful to maintain data size while reducing outlier effects.
  - **Statistical Impute:** Mean/Mode/Median
  - **Impute with model**
  - **Domain-Specific Impute**
- **Transform**: (e.g., log transformation) Reduces the impact of extreme values

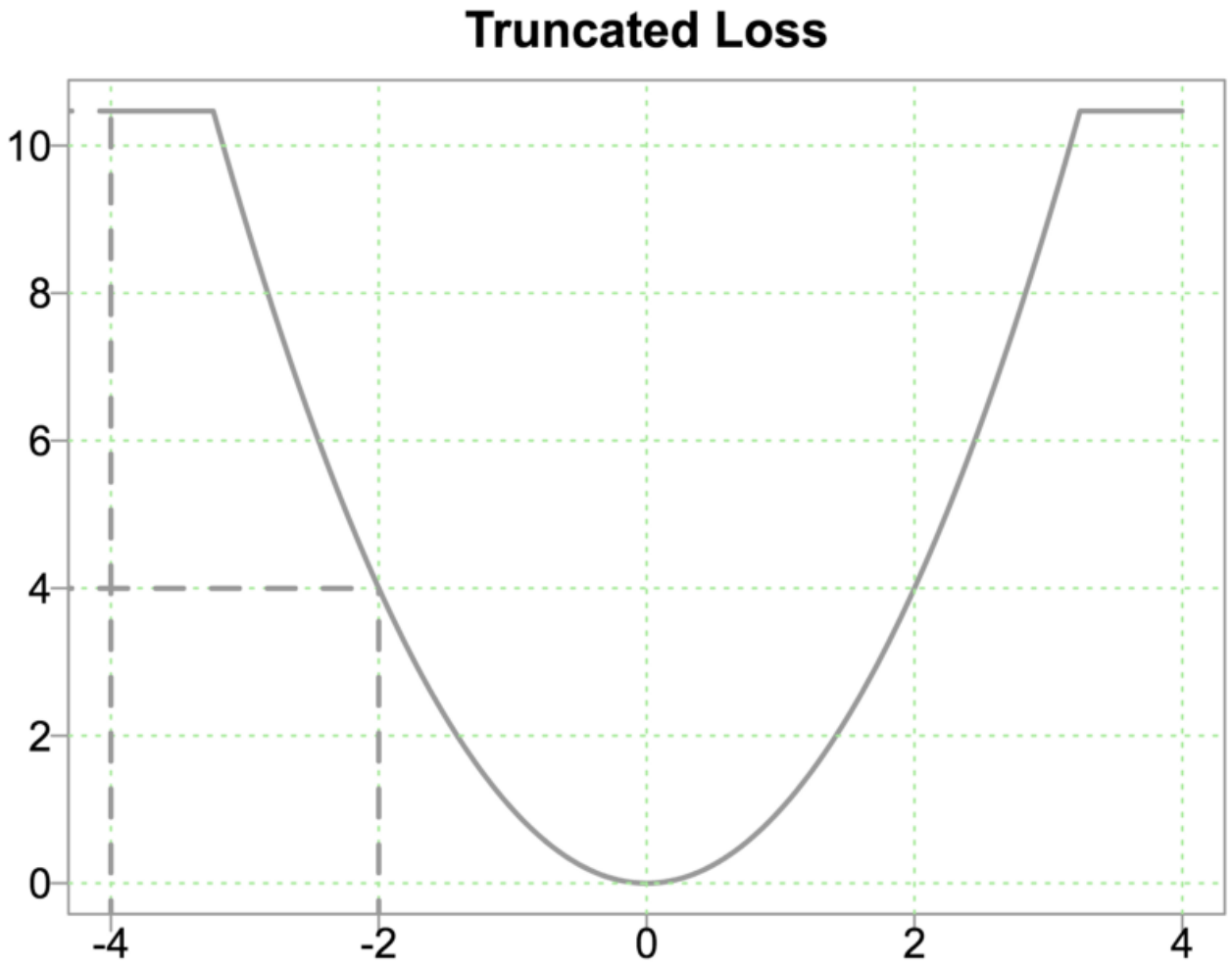
Remove VS Keep:

- Remove if the outliers are due to errors or irrelevant.
- Keep if the outliers are informative, such as fraud detection



Handling Outliers: Metric Change

$$L(e) = \begin{cases} e^2 & \forall\{e;\ Q_{.025}(e) < e < Q_{.0975}(e)\} \\ constant & \text{otherwise.} \end{cases},$$



Live Coding & Demos #2

Transformation and Encoding

Why Transformation and Encoding Matter?

- **Data Transformation & Encoding:**
  Converting data to prepare for model input.

- **Importance**: Ensures features are consistent and interpretable.

- **Examples**:
  Predicting house prices: scaling numerical features (e.g., area) and encoding categorical variables (e.g., location).

Scaling and Normalization

Min-Max Scaling (Normalization):

- **Definition**: Scales values between 0 and 1.
- **Use Case**: Algorithms sensitive to distance, like **KNN** or **SVM**.
- **Code Example**: MinMaxScaler from Scikit-learn.

Standardization:

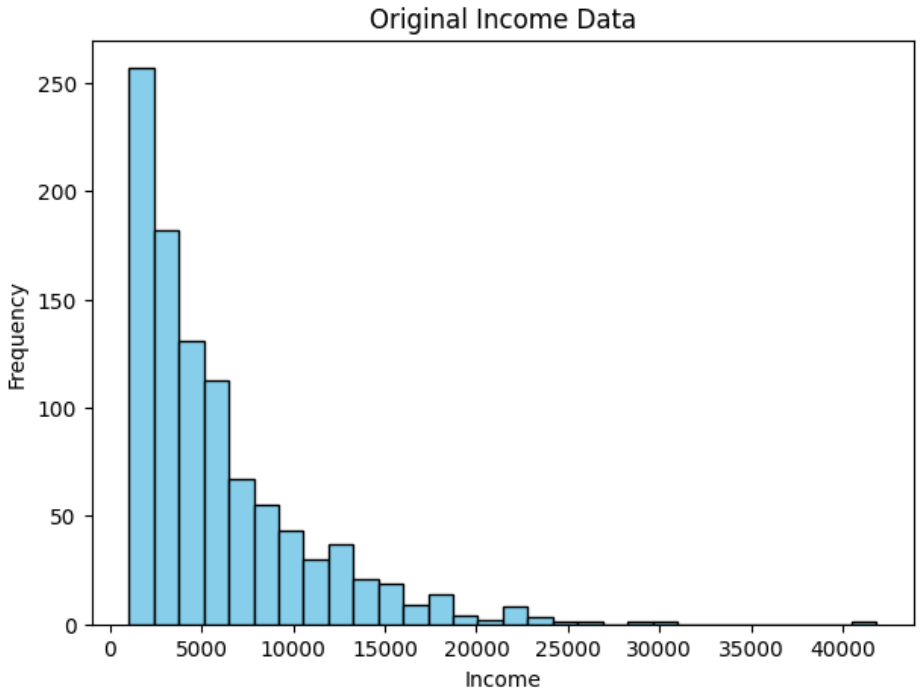
- **Definition**: Converts features to have a mean of 0 and standard deviation of 1.
- **Use Case**: Ideal for normally distributed data.
- **Code Example**: StandardScaler from Scikit-learn.

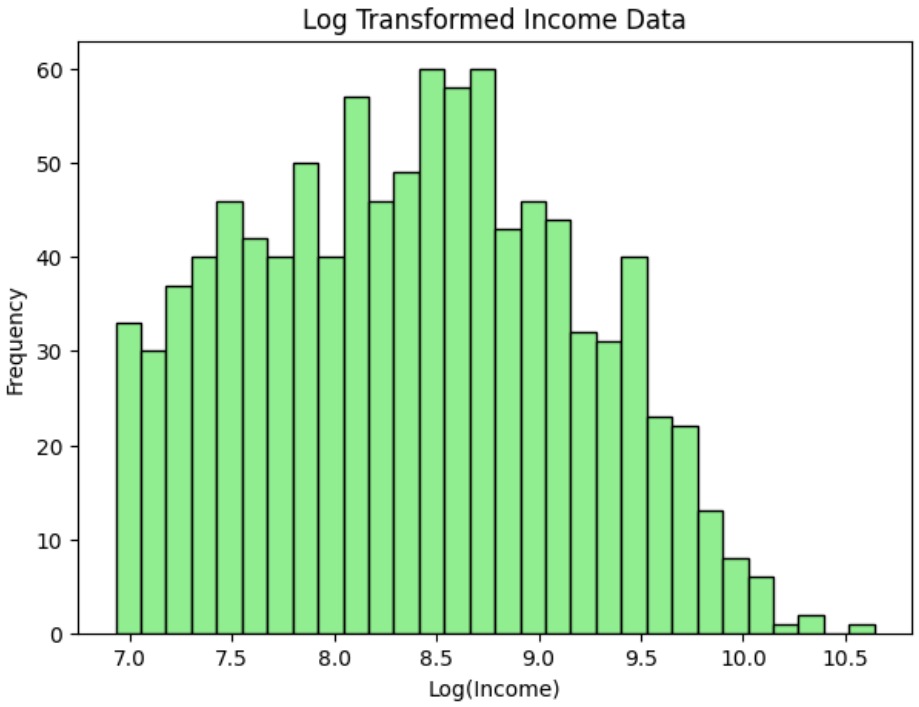
Robust Scaling:

- **Definition**: Uses median and IQR, good for data with **outliers**.
- **Use Case**: Handling data with many outliers.
- **Code Example**: RobustScaler from Scikit-learn.

Log Transformation

- **Log Transformation:**
  - Applies logarithm to compress large values. Reduces skewness, especially for right-skewed data. Use for large, positive, right-skewed values.
  - Example: Transforming sales data to stabilize variance.





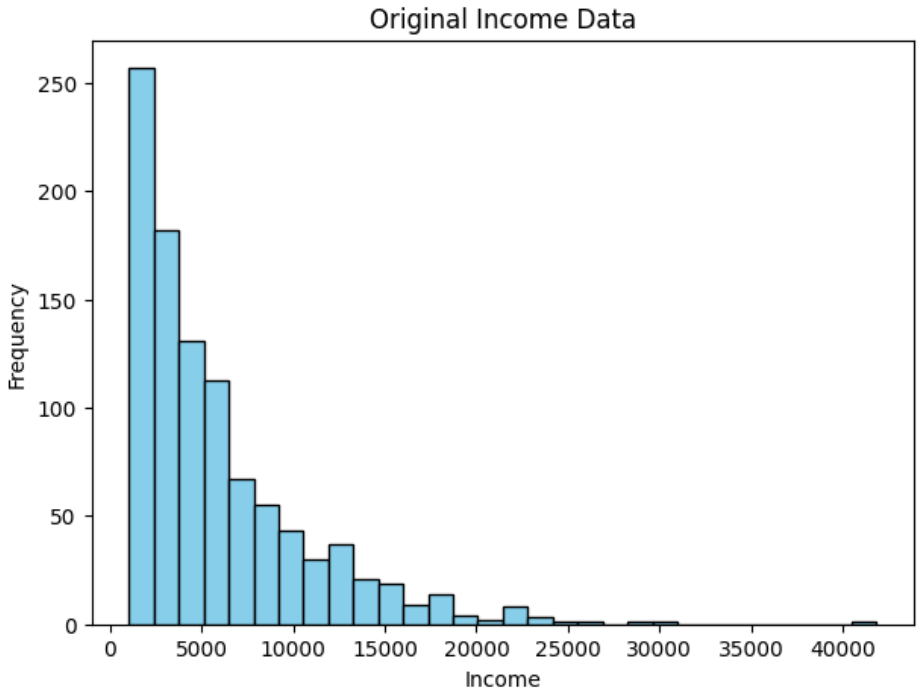
<u>Practical Tip</u>: Always visualize the effects of your transformation.

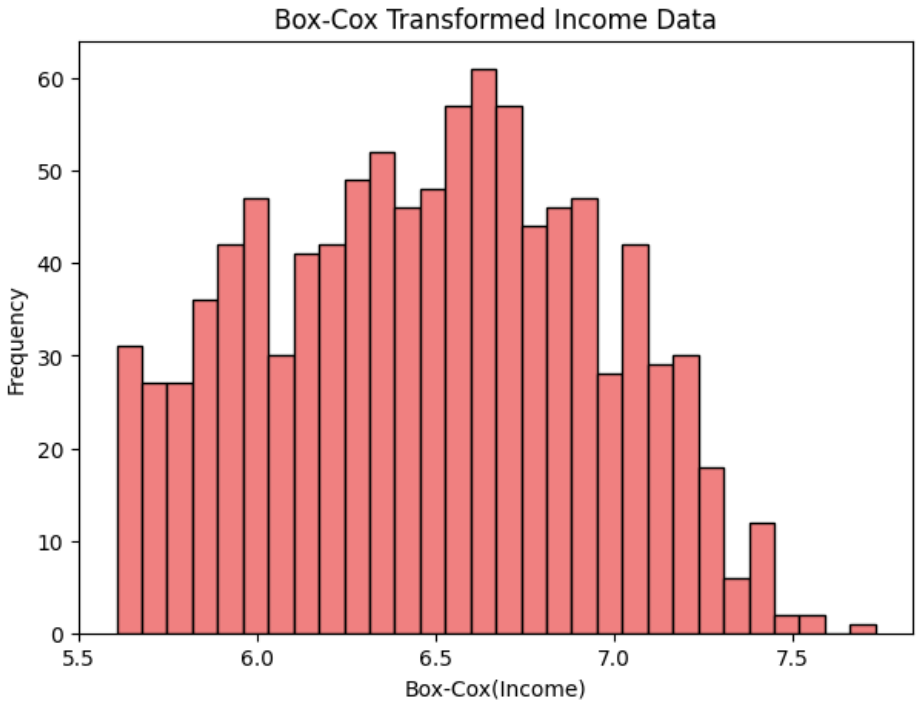
Box-Cox Transformation

Definition: A power transformation that makes data more normally distributed.

Use Case: Handling data with both positive and small negative values.

When data can be positive or small negative and needs normalization.



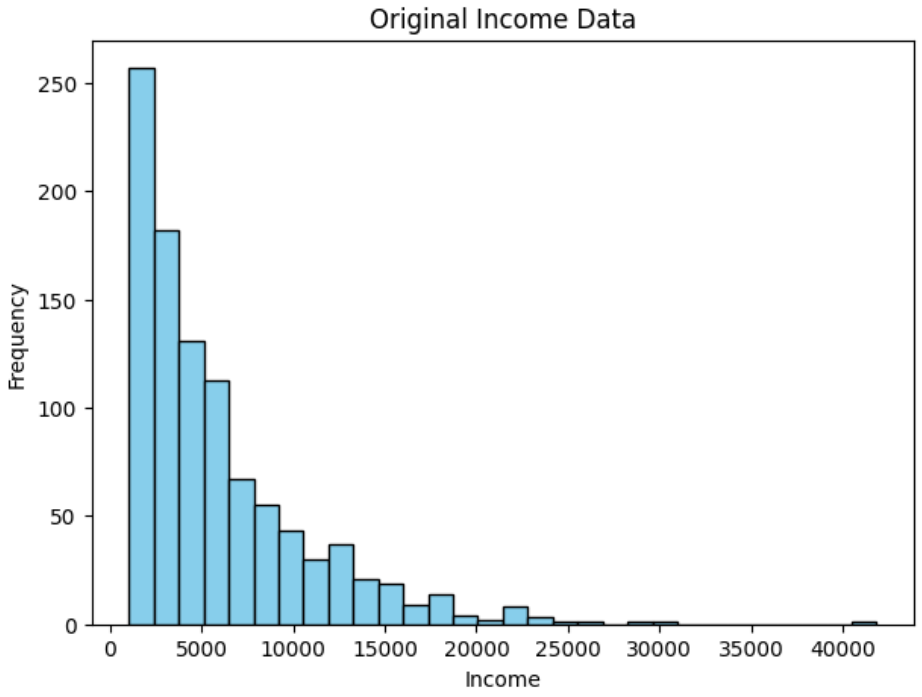


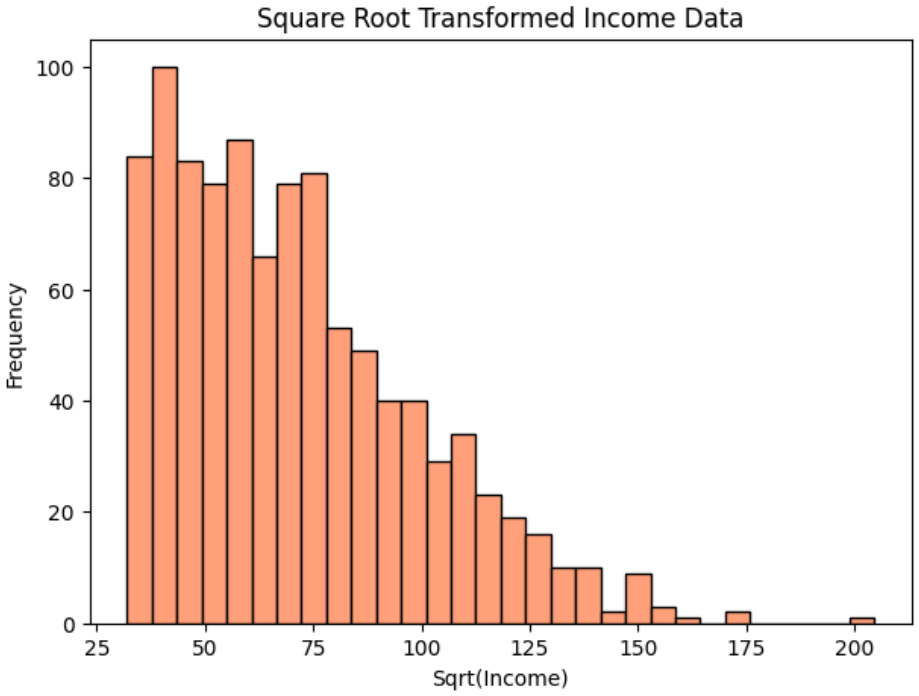
<u>Practical Tip</u>: Always visualize the effects of your transformation.

Square Root Transformation

Definition: Reduces the impact of outliers, especially for small positive numbers.

Use Case: Small positive numbers like counts, scores, or item sales.



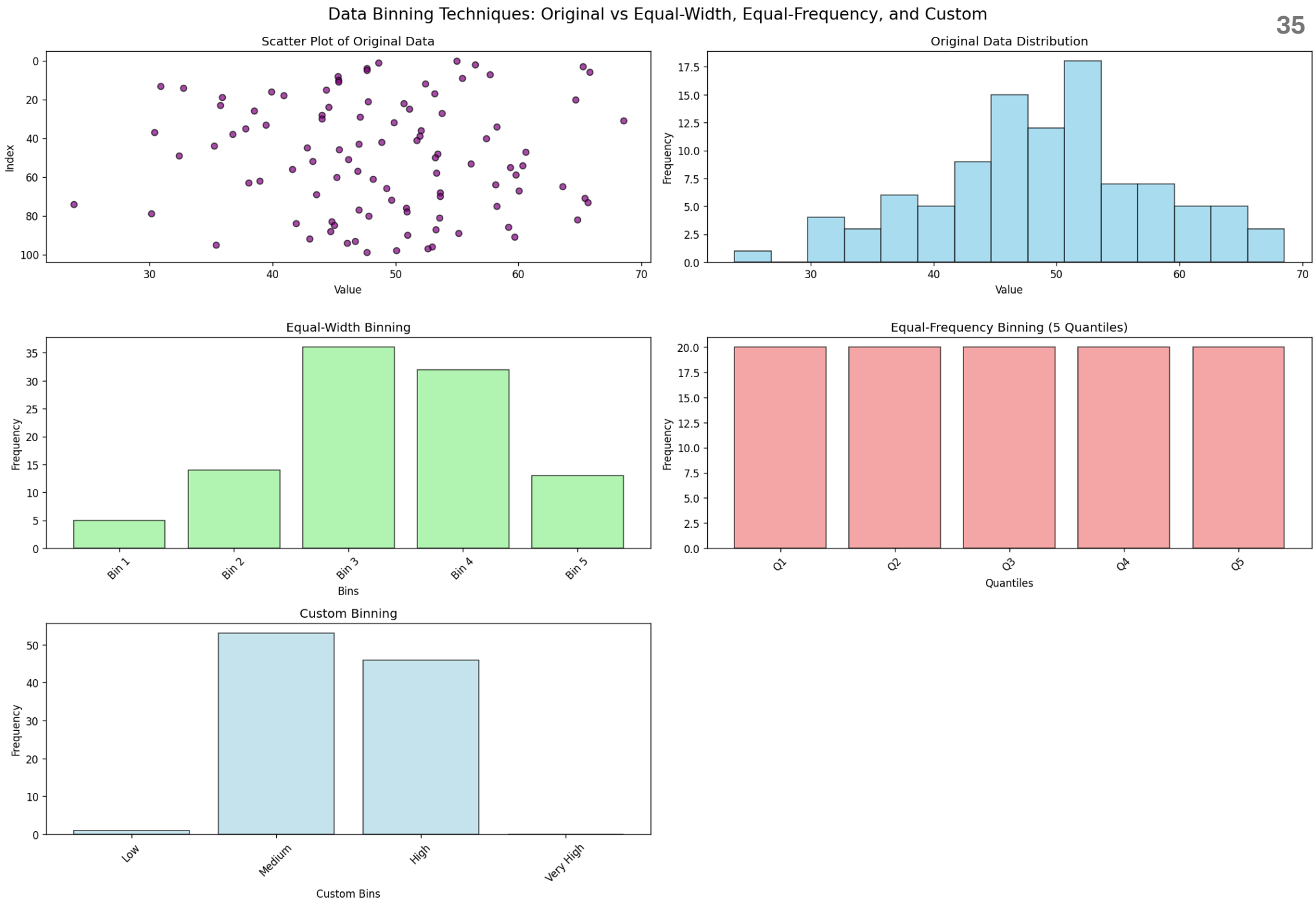


<u>Practical Tip</u>: Always visualize the effects of your transformation.

Data Binning

- **Definition:** Converting continuous numerical data into discrete bins or intervals.
- Binning Types:
  - **Equal-Width**: Divides the range of values into bins of equal size.
  - **Equal-Frequency**: Each bin has approximately the same number of data points.
  - **Quantile**: Divides data based on quantiles, creating bins with equal probabilities.
  - **Custom**: User-defined bins based on domain knowledge.
- **Benefits**: Simplifies data, reduces model complexity, improves interpretability.
- **Challenges**: May lead to information loss, artificial cut-off points.
- **Consideration**: Use domain knowledge to define meaningful bins.
- **Example:** Binning ages into categories: 0-18 (Youth), 19-35 (Young Adult), 36-60 (Adult), 60+ (Senior).

Binning Graphs



Why Data Encoding is Crucial for Machine Learning?

- **Definition**: Converting categorical data to numerical format.
- **Importance**: Allows machine learning algorithms to use features that are non-numeric.
- **Examples**: Car color, city name, or gender.

Overview of Encoding Techniques

- **One-Hot Encoding:** Simple, creates binary columns for each category.
- **Label Encoding:** Assigns a unique number to each category.
- **Ordinal Encoding:** Ordered categories, maintains rank.
- **Target Encoding:** Replaces categories with the mean of the target variable.
- **Frequency Encoding:** Replaces categories with frequency counts.
- **Binary Encoding:** Hybrid of label and one-hot encoding to reduce dimensionality.

One-Hot Encoding

Definition: Creates binary columns for each unique category.

Example: Color feature ('Red', 'Blue', 'Green') becomes separate columns for each color.

Pros: Simple and easy to interpret.

Cons: Can lead to high dimensionality.

workclass
State-gov
Self-emp-not-inc
Private
Private
Private

State-gov	Self-emp-not-inc	Private
1	0	0
0	1	0
0	0	1
0	0	1
0	0	1

Label Encoding

Definition: Assigns each category a unique numerical label.

Example: Animal type ('Cat', 'Dog', 'Bird') becomes 0, 1, 2.

Pros: Simple and memory-efficient.

Cons: Implies an ordinal relationship, which can mislead models.

workclass
State-gov
Self-emp-not-inc
Private
Private
Private

workclass
0
1
2
2
2

Ordinal Encoding

- **Definition:** Similar to Label Encoding, but *the categories have an inherent order*
- **Use Case**: Ordered categories like satisfaction levels.
- **Example**: Satisfaction ('Low', 'Medium', 'High') becomes 0, 1, 2.
- **Pros**: Retains the ordinal relationship.

workclass
State-gov
Self-emp-not-inc
Private
Private
Private

workclass
0
1
2
2
2

Target Encoding

- **Definition**: Replaces categories with the mean / median / moda of the target variable.
- **Use Case**: High cardinality features (e.g., zip codes).
- **Cons**: Can lead to overfitting if not handled properly.

workclass	target
State-gov	0
Self-emp-not-inc	1
Private	0
Private	0
Private	1

workclass	target mean
State-gov	0
Self-emp-not-inc	1
Private	1/3

workclass
0
1
1/3
1/3
1/3

Frequency Encoding

- **Definition**: Replaces categories with their frequency counts.
- **Use Case**: Efficient for categorical variables with many levels.
- **Pros**: Low dimensionality.

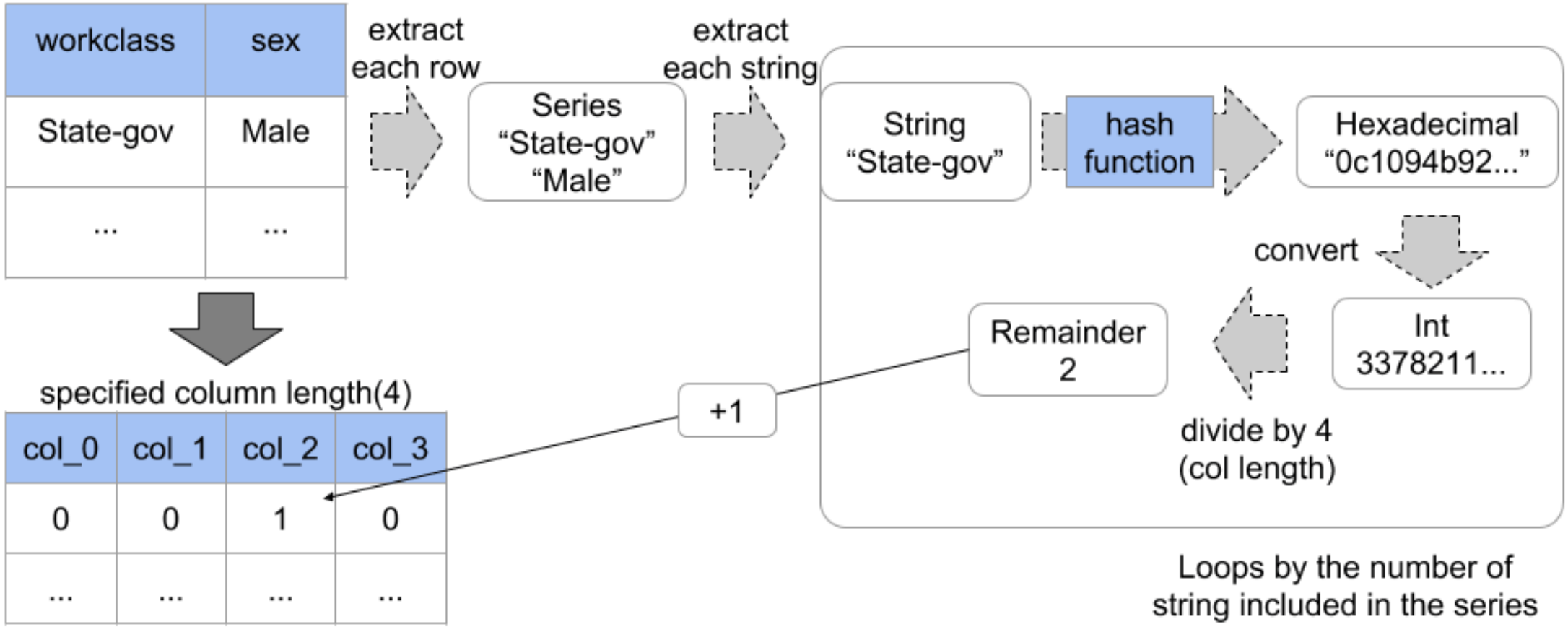
Binary Encoding

- **Definition**: Combines Label Encoding and One-Hot Encoding to reduce dimensionality. It assigns each category a number, then converts that number to binary, and splits it across different columns.
- **Use Case**: Efficient for high cardinality features without creating too many columns.
- **Example**: Category '5' as binary '101', split across columns.

Other techniques…

- **BaseN:** Similar to binary encoding but converts category indices into any base. It is useful for compact representation.
- **Hashing:** Maps categories into a fixed number of columns using hashing. This is efficient for very high-cardinality features but can result in hash collisions.
- **Leave-One-Out:** Similar to target encoding but excludes the current observation when calculating the mean to reduce overfitting.
- **WOE (Weight of Evidence)**: Commonly used in financial risk modeling; replaces categories based on their predictive power using log-odds of the target variable.
- **Backward Difference:** Compares each category with the previous level, capturing differences. It's useful for models that are sensitive to contrasts between categories.
- **Helmert:** Captures contrasts between the current level and the mean of subsequent levels. Useful for interpreting differences between categories.

Hash Encoding Illustration



Summary of Basic Encoding Techniques

Encoding Technique	Pros	Cons	Best Use Case
One-Hot Encoding	Simple, easy to interpret.	Can lead to high dimensionality with many categories.	Low cardinality features, such as 'Gender'.
Label Encoding	Memory efficient, fast.	Implies an ordinal relationship, which may be misleading.	Ordinal categories like 'Low', 'Medium', 'High'.
Ordinal Encoding	Retains the ordinal relationship.	Bias if order doesn't accurately represent data.	Features with a defined order, e.g., product ratings.

Summary of Advanced Encoding Techniques

Encoding Technique	Pros	Cons	Best Use Case
Target Encoding	Reduces dimensionality significantly.	Risk of overfitting.	High cardinality features, e.g., user IDs in churn prediction.
Frequency Encoding	Low dimensionality, fast to compute.	Can lead to overfitting if frequency is directly correlated with target.	High cardinality features such as zip codes.
Binary Encoding	Reduces dimensionality compared to One-Hot Encoding.	Slightly more complex to implement.	High cardinality data with numerous categories.

I CANNOT TAKE THIS BORING
PRESENTATION ANYMORE

Live Coding & Demos #3

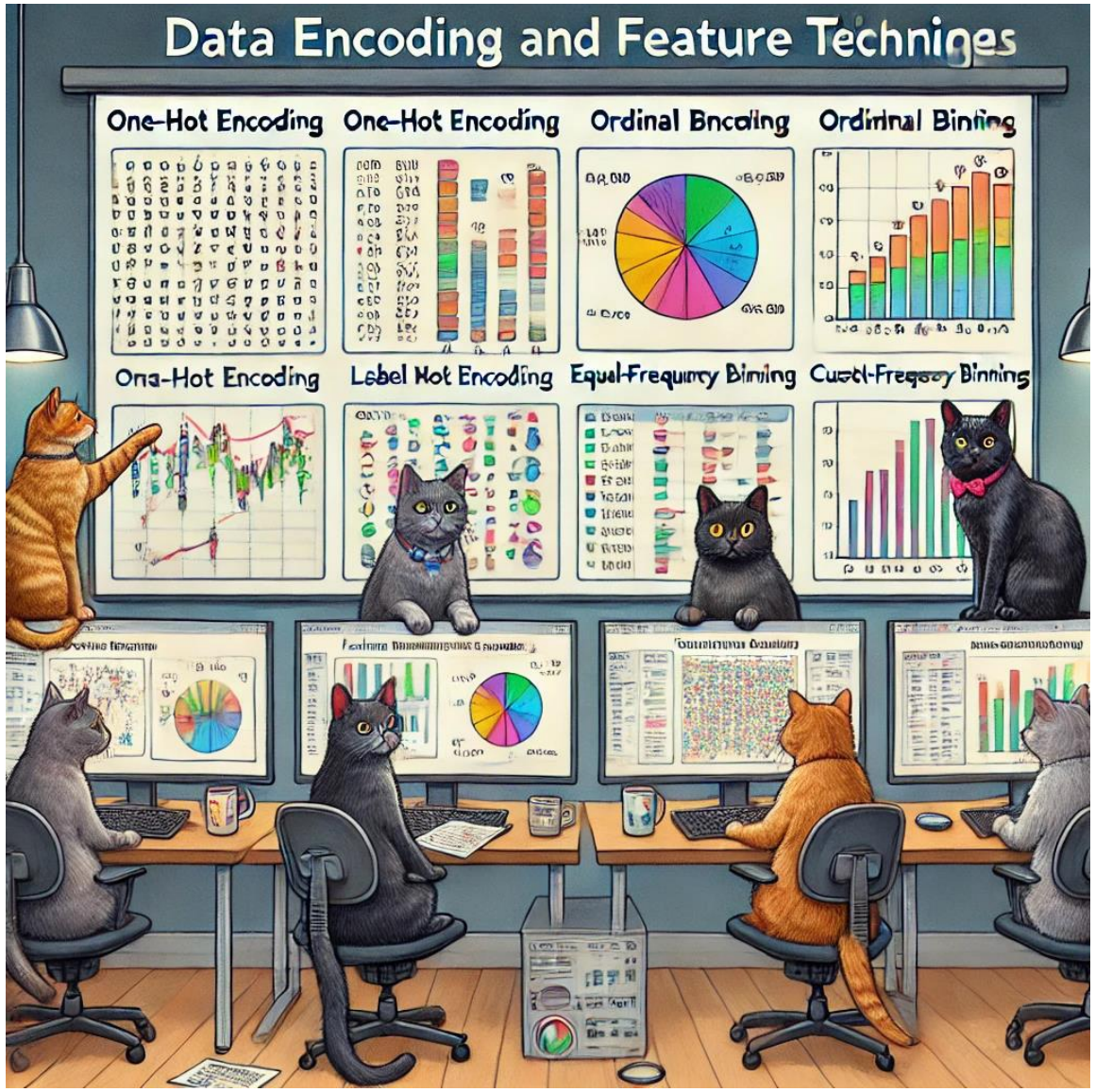
Common Data Preprocessing Challenges

Practical Considerations in Real-World Data Preparation

- **Ethics in Data Preparation**: Bias handling, fairness, and transparency.
- **Handling Missing Values**: Missing data can lead to biased models.
- **Outliers**: Extreme values can distort model performance.
- **Mixed Data Types**: Handling both numerical and categorical data requires thoughtful preprocessing and encoding.
- **Scalability**: Managing large datasets that cannot fit in memory requires efficient preprocessing strategies including memory management, distributed processing, and batch processing or sampling.

Summary and Wrap-Up

Wrapping Up - Key Takeaways & Final Thoughts

- **Key Takeaways**:
  - **Data Cleaning:** Handle missing values, remove duplicates, and correct inconsistencies.
  - **Encoding:** Choose the right technique based on the data type and model requirements.
  - **Outlier Management:** Identify and handle outliers to maintain model robustness.
  - **Transformation & Binning:** Apply these techniques to ensure consistent scaling and improved interpretability.
- **Final Thoughts**:
  - Good data preparation reduces noise and provides a strong base for modeling.
  - Always experiment with different techniques to see which best fits your dataset.

Thank you for your attention